

MEMAHAMI OAuth 2.0

Panduan Terstruktur untuk Pengembang Backend

Dari Masalah Nyata Menuju Solusi Protokol

Disusun dalam Bahasa Indonesia

Daftar Isi

Daftar Isi.....	2
Kata Pengantar	4
Bab 1 — Masalah Sebelum OAuth	5
1.1 Pola Berbahaya: Berbagi Kata Sandi	5
Bab 2 — Apa Itu OAuth 2.0 (dan Apa yang Bukan).....	6
2.1 Autentikasi vs. Otorisasi	6
Bab 3 — Registrasi Aplikasi dan Asal-Usul Kredensial.....	7
Bab 4 — Empat Peran dalam OAuth 2.0	8
Bab 5 — Access Token dan Scope	9
5.1 Apa Itu Scope?	9
Bab 6 — Alur Authorization Code	10
Langkah 1 — Pengguna Memulai Aksi	10
Langkah 2 — JalanKita Menyusun URL Pengalihan	10
Langkah 3 — Rani Melakukan Autentikasi dan Persetujuan.....	10
Langkah 4 dan 5 — Google Menerbitkan Authorization Code	10
Langkah 6 dan 7 — JalanKita Menukar Kode dengan Token.....	11
Langkah 8 dan 9 — Memanggil API.....	11
Bab 7 — Mengapa Ada Dua Langkah Redirect	12
Bab 8 — Kedaluwarsa Token dan Refresh Token	13
8.1 Cara Kerja Refresh Token	13
Bab 9 — PKCE: Melindungi Public Client.....	14
9.1 Cara Kerja PKCE.....	14
Bab 10 — state vs. PKCE: Melawan Serangan yang Berbeda	15
Bab 11 — OAuth 2.0 vs. OpenID Connect (OIDC) dan JWT.....	16
11.1 Kehadiran OpenID Connect (OIDC)	16
11.2 Apa Itu JWT?	16
11.3 Apakah Access Token Selalu Berbentuk JWT?	16
Bab 12 — Mengetahui OAuth 2.1	18
Bab 13 — Kesalahan Keamanan Umum di Lingkungan Produksi	19
13.1 Menyimpan Token di localStorage Browser	19
13.2 Membocorkan Client Secret	19
13.3 Meminta Scope yang Tidak Perlu (Scope Creep)	19
13.4 Menganggap Token OAuth Sebagai Bukti Identitas	19
13.5 Melewatkan Validasi state atau PKCE	19
Bab 14 — Penutup	21
Glosarium.....	22

Referensi	22
------------------------	-----------

Kata Pengantar

Jika Anda bertanya kepada sepuluh pengembang backend tentang cara kerja OAuth 2.0, sebagian besar akan langsung menyebutkan istilah-istilah seperti “Authorization Server”, “Bearer Token”, “PKCE”, atau “Implicit Grant”. Sebagian bahkan bisa menggambarkan diagram alur dengan banyak anak panah yang saling silang. Namun ketika ditanya mengapa satu permintaan HTTP tertentu harus ada, atau apa yang akan rusak jika langkah itu dihilangkan, jawaban yang jelas sering kali sulit didapat.

Hal ini terjadi karena OAuth kerap diajarkan secara terbalik: dimulai dari definisi dan diagram, bukan dari alasan mengapa protokol ini dirancang seperti itu sejak awal. Buku ini disusun untuk memperbaiki pendekatan tersebut.

Kita akan membangun pemahaman tentang OAuth 2.0 selangkah demi selangkah, dimulai dari sebuah persoalan rekayasa perangkat lunak yang nyata, lalu tiba secara alami pada solusi-solusi yang ditawarkan oleh protokol ini. Setiap parameter, setiap pengalihan (redirect), dan setiap token akan masuk akal karena Anda memahami masalah yang coba dipecahkannya, bukan sekadar menghafal istilah.

Buku ini disusun berdasarkan alur pembahasan pada artikel “How OAuth 2.0 Works: A Practical Guide for Backend Developers” oleh Ashutosh Krishna yang dipublikasikan di freeCodeCamp, yang kemudian diuraikan ulang, diperluas, dan disusun dalam Bahasa Indonesia dengan struktur bab yang lebih formal agar dapat dijadikan bahan belajar mandiri maupun referensi tim.

Selamat membaca, dan selamat memahami OAuth 2.0 dari akar masalahnya.

Bab 1 — Masalah Sebelum OAuth

Bayangkan kita sedang membangun sebuah aplikasi bernama JalanKita, sebuah aplikasi backend yang membantu pengguna merencanakan perjalanan. Salah satu fitur unggulannya adalah mendeteksi bentrok jadwal secara otomatis dan menambahkan rencana perjalanan langsung ke Google Calendar milik pengguna.

Agar fitur ini bisa bekerja, JalanKita membutuhkan akses ke Google Calendar API. Secara spesifik, aplikasi ini perlu membaca jadwal yang sudah ada dan menuliskan jadwal baru atas nama pengguna kita, sebut saja namanya Rani.

Pertanyaannya: bagaimana kita akan menyelesaikan kebutuhan ini pada masa sebelum OAuth ada?

1.1 Pola Berbahaya: Berbagi Kata Sandi

Tanpa adanya protokol seperti OAuth, satu-satunya cara yang terpikirkan adalah meminta Rani memasukkan langsung username dan password akun Google miliknya ke dalam formulir JalanKita. Password tersebut kemudian disimpan di basis data JalanKita, dan digunakan setiap kali aplikasi perlu mengambil atau menambahkan jadwal ke Google Calendar.

Pendekatan ini memang bisa “berjalan” secara teknis, tetapi menimbulkan sejumlah masalah keamanan dan operasional yang serius:

1. Akses yang berlebihan (over-privileged access). JalanKita sebenarnya hanya butuh mengelola jadwal kalender. Namun karena memegang password asli akun Google Rani, aplikasi ini secara teknis juga bisa membaca email di Gmail, melihat isi Google Drive, menghapus foto, bahkan mengganti password akun tersebut. Tidak ada cara untuk membatasi akses JalanKita hanya pada satu layanan saja.
2. Tidak ada kontrol pencabutan akses yang granular. Jika suatu saat Rani ingin menghentikan akses JalanKita ke kalendernya, satu-satunya cara adalah mengganti password akun Google-nya. Tindakan ini otomatis memutus akses semua aplikasi lain yang sebelumnya sudah diizinkan olehnya.
3. Beban tanggung jawab penyimpanan bagi JalanKita. JalanKita kini menyimpan password — baik dalam bentuk teks polos maupun terenkripsi yang bisa didekripsi kembali — milik ribuan akun Google penggunanya. Satu kebocoran basis data atau celah SQL injection saja sudah cukup untuk membongkar kunci utama ke seluruh kehidupan digital penggunanya di Google.
4. Normalisasi kebiasaan phishing. Membiasakan pengguna memasukkan kredensial utama Google mereka ke aplikasi pihak ketiga mengajarkan kebiasaan keamanan yang buruk dan membuat mereka lebih rentan terhadap serangan phishing di masa depan.

Yang sebenarnya kita butuhkan adalah cara bagi Rani untuk memberi izin kepada JalanKita agar dapat melakukan tindakan tertentu pada Google Calendar-nya, tanpa pernah menyerahkan password akun Google-nya sama sekali.

Kemampuan semacam ini disebut *delegated authorization* (otorisasi terdelegasi), dan inilah yang menjadi inti dari apa yang ditawarkan OAuth 2.0.

Bab 2 — Apa Itu OAuth 2.0 (dan Apa yang Bukan)

OAuth 2.0 adalah sebuah standar terbuka untuk otorisasi terdelegasi. Protokol ini menyediakan kerangka kerja yang memungkinkan seorang pengguna memberi izin kepada aplikasi pihak ketiga untuk mengakses sumber dayanya di layanan lain, tanpa harus membagikan kredensial login miliknya.

Sebelum melangkah lebih jauh, penting untuk meluruskan salah kaprah paling umum dalam pengembangan aplikasi web.

2.1 Autentikasi vs. Otorisasi

Kedua istilah ini sering dipakai bergantian secara keliru, padahal keduanya menjawab pertanyaan yang sangat berbeda:

- Autentikasi (AuthN): Siapa Anda? — berkaitan dengan identitas.
- Otorisasi (AuthZ): Apa yang boleh Anda lakukan? — berkaitan dengan izin/hak akses.

Autentikasi selalu mendahului otorisasi: sistem terlebih dahulu memastikan “Anda adalah Rani”, baru kemudian menentukan “Rani boleh membaca dan menulis jadwal, tetapi tidak boleh menghapus seluruh kalender”.

OAuth 2.0 murni merupakan kerangka kerja otorisasi.

Protokol ini tidak mengatur bagaimana cara mengautentikasi pengguna, bagaimana menerbitkan detail identitas, atau bagaimana menyimpan akun pengguna. OAuth 2.0 hanya berurusan dengan penerbitan “kunci izin” (token) agar satu layanan dapat berbicara dengan layanan lain atas nama pengguna.

Saat Anda menekan tombol “Masuk dengan Google” di sebuah situs, interaksi tersebut sebenarnya menggunakan lapisan tambahan di atas OAuth yang disebut OpenID Connect (OIDC), yang akan dibahas lebih lanjut pada Bab 11. Namun inti dari OAuth 2.0 itu sendiri sepenuhnya berkaitan dengan otorisasi, bukan autentikasi.

Bab 3 — Registrasi Aplikasi dan Asal-Usul Kredensial

Sebelum JalanKita dapat memulai proses OAuth apa pun, aplikasi ini harus terlebih dahulu didaftarkan di konsol pengembang milik penyedia layanan — dalam kasus kita, Google Cloud Console.

Pada saat pendaftaran, Google akan meminta sejumlah informasi penting dari JalanKita:

5. Nama dan logo aplikasi, yang nantinya akan ditampilkan pada layar persetujuan (consent screen) yang dilihat pengguna.
6. Redirect URI, yaitu alamat callback yang tepat (misalnya <https://jalankita.id/login/oauth2/code/google>) ke mana Google diizinkan mengirim kode otorisasi setelah pengguna menyetujui akses.

Setelah proses registrasi selesai, Google akan menerbitkan dua kredensial penting untuk JalanKita:

- `client_id` — sebuah identitas publik (mirip nama pengguna) yang mengidentifikasi JalanKita. Nilai ini aman untuk disematkan pada tautan publik atau kode di sisi frontend.
- `client_secret` — sebuah kunci rahasia (mirip password) yang digunakan oleh server backend JalanKita untuk membuktikan identitasnya saat menukarkan kode otorisasi dengan token.

Ingat baik-baik: `client_id` boleh diketahui publik, tetapi `client_secret` harus selalu dijaga kerahasiaannya dan hanya boleh disimpan di lingkungan server yang tepercaya.

Bab 4 — Empat Peran dalam OAuth 2.0

OAuth 2.0 mendefinisikan empat peran utama. Mari kita petakan langsung ke dalam contoh JalanKita agar istilah-istilah ini tidak lagi terasa abstrak.

Peran	Definisi	Contoh pada JalanKita
Resource Owner	Pengguna yang memiliki data.	Rani, pemilik akun Google Calendar.
Client	Aplikasi pihak ketiga yang ingin mengakses data pengguna.	JalanKita (backend Spring Boot / Node.js kita).
Authorization Server	Server yang mengautentikasi pengguna, meminta persetujuan, dan menerbitkan access token.	Server OAuth milik Google (accounts.google.com).
Resource Server	Server yang menyimpan data pengguna yang dilindungi.	Google Calendar API (www.googleapis.com/calendar).

Tabel 4.1 — Pemetaan empat peran OAuth 2.0 ke dalam studi kasus JalanKita.

Perhatikan bahwa tanggung jawab Google terbagi menjadi dua peran terpisah: Authorization Server (yang menerbitkan token) dan Resource Server (yang menyediakan API). Pada organisasi besar, kedua peran ini sering kali dikelola oleh tim yang benar-benar berbeda, meskipun keduanya berada di bawah payung perusahaan yang sama.

Bab 5 — Access Token dan Scope

Alih-alih menyerahkan password-nya, Rani cukup menyetujui penerbitan sebuah access token. Access token adalah sederet karakter yang berfungsi layaknya kartu akses sementara. Setiap kali JalanKita mengirim permintaan HTTP ke Google Calendar, token inilah yang disertakan pada header permintaan tersebut.

Access token memiliki tiga sifat penting yang tidak dimiliki oleh password:

7. Cakupan terbatas (limited scope) — hanya dapat digunakan untuk izin-izin tertentu yang telah disepakati.
8. Umur terbatas (limited lifetime) — kedaluwarsa secara otomatis setelah periode waktu yang singkat, biasanya dalam hitungan menit hingga jam.
9. Dapat dicabut (revocable) — baik Rani maupun Google dapat mencabut token ini kapan saja tanpa memengaruhi password akun Rani.

5.1 Apa Itu Scope?

Scope adalah sebuah string yang mendefinisikan izin spesifik yang diminta. Alih-alih meminta “akses ke akun Google” secara umum, JalanKita meminta scope yang spesifik, misalnya:

- Membaca jadwal kalender: <https://www.googleapis.com/auth/calendar.events.readonly>
- Membuat/mengubah jadwal kalender: <https://www.googleapis.com/auth/calendar.events>

Ketika JalanKita mengarahkan Rani ke Google, scope yang diminta tersebut ditampilkan secara eksplisit dalam layar persetujuan, misalnya dengan kalimat: “JalanKita ingin izin untuk melihat dan mengubah jadwal Google Calendar Anda.”

Jika Rani menyetujuinya, access token yang diterbitkan Google akan terikat ketat pada scope yang diminta tersebut. Apabila JalanKita mencoba menggunakan token yang sama untuk membaca email Rani, Resource Server Google akan menolak permintaan tersebut dengan status HTTP 403 Forbidden.

Bab 6 — Alur Authorization Code

Setelah memahami peran dan token, pertanyaan berikutnya adalah: bagaimana caranya JalanKita benar-benar mendapatkan access token tersebut?

Alur paling standar dan paling aman untuk aplikasi sisi server (seperti backend JalanKita kita) adalah Authorization Code Flow. Berikut adalah rangkaian langkahnya, yang akan kita telaah satu per satu.

Langkah 1 — Pengguna Memulai Aksi

Rani sedang menggunakan antarmuka JalanKita dan menekan tombol “Hubungkan Google Calendar.”

Langkah 2 — JalanKita Menyusun URL Pengalihan

Backend JalanKita tidak meminta kredensial apa pun dari Rani. Sebaliknya, ia membangun sebuah URL menuju Authorization Server milik Google dan mengarahkan browser Rani ke sana. URL tersebut kurang lebih berbentuk sebagai berikut:

```
GET https://accounts.google.com/o/oauth2/v2/auth?
  response_type=code&
  client_id=JALANKITA_CLIENT_ID&
  redirect_uri=https://jalankita.id/login/oauth2/code/google&
  scope=https://www.googleapis.com/auth/calendar.events&
  state=xyz123
```

Mari kita telaah fungsi masing-masing parameter:

- `response_type=code` — memberi tahu Google bahwa kita menggunakan alur Authorization Code.
- `client_id` — identitas publik yang diberikan Google kepada JalanKita saat pendaftaran.
- `redirect_uri` — alamat tujuan pengalihan Rani setelah proses persetujuan selesai.
- `scope` — izin-izin yang diminta oleh JalanKita.
- `state` — string acak yang dibuat oleh JalanKita untuk mencegah serangan Cross-Site Request Forgery (CSRF).

Langkah 3 — Rani Melakukan Autentikasi dan Persetujuan

Browser Rani kini berada di domain Google (`accounts.google.com`). Google memeriksa apakah Rani sudah login; jika belum, Google akan memintanya login terlebih dahulu. JalanKita sama sekali tidak melihat proses ini — inilah salah satu poin krusial dari desain OAuth.

Setelah berhasil login, Google menampilkan layar persetujuan yang mencantumkan nama JalanKita beserta scope yang diminta.

Langkah 4 dan 5 — Google Menerbitkan Authorization Code

Setelah Rani menekan “Setujui,” Authorization Server Google mengarahkan kembali browser Rani ke `redirect_uri` milik JalanKita, dengan menyertakan sebuah Authorization Code berumur pendek beserta parameter `state` pada query string:

```
GET https://jalankita.id/login/oauth2/code/google?
code=4/0AX4XfWh...&state=xyz123
```

Backend JalanKita memverifikasi bahwa nilai state yang diterima cocok dengan yang dikirimkan sebelumnya. Jika cocok, JalanKita menggunakan nilai code tersebut untuk langkah selanjutnya.

Langkah 6 dan 7 — JalanKita Menukar Kode dengan Token

Server backend JalanKita — bukan browser — kini melakukan permintaan POST langsung ke endpoint token milik Google (<https://oauth2.googleapis.com/token>):

```
POST /token HTTP/1.1
Host: oauth2.googleapis.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&
code=4/0AX4XfWh...&
redirect_uri=https://jalankita.id/login/oauth2/code/google&
client_id=JALANKITA_CLIENT_ID&
client_secret=JALANKITA_CLIENT_SECRET
```

Google memvalidasi kode otorisasi tersebut beserta client_secret milik JalanKita. Jika valid, Google merespons dengan payload JSON berisi access token:

```
{
  "access_token": "ya29.a0ARrdaM...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "1//04rG...",
  "scope": "https://www.googleapis.com/auth/calendar.events"
}
```

Langkah 8 dan 9 — Memanggil API

JalanKita menyimpan access token ini secara aman lalu menggunakannya untuk memanggil Google Calendar API atas nama Rani:

```
GET /calendar/v3/users/me/calendarList HTTP/1.1
Host: www.googleapis.com
Authorization: Bearer ya29.a0ARrdaM...
```

Google Calendar menerima permintaan tersebut, mengekstrak Bearer token dari header, memeriksa validitas serta kecocokan scope-nya melalui infrastruktur otentikasi Google, lalu mengembalikan data kalender milik Rani.

Bab 7 — Mengapa Ada Dua Langkah Redirect

Pada titik ini, pengembang pemula hampir selalu mengajukan pertanyaan bagus:

“Mengapa harus ada Langkah 4 dan Langkah 6? Mengapa Google tidak langsung mengembalikan access token pada pengalihan di Langkah 4?”

Mengapa harus repot-repot mengembalikan `authorization_code` sementara ke browser, hanya untuk kemudian melakukan panggilan backend terpisah guna menukarnya dengan `access_token` yang sebenarnya?

Jawabannya terletak pada perbedaan antara Front-Channel dan Back-Channel.

- **Front-Channel (browser):** Browser adalah lingkungan yang tidak sepenuhnya tepercaya dan sangat terekspos. URI pengalihan dapat tersimpan dalam riwayat browser, log sistem, header referrer, bahkan dapat dibaca oleh ekstensi browser. Jika Google langsung mengembalikan access token melalui URL browser, token bernilai tinggi tersebut sangat berisiko bocor atau dicegat oleh ekstensi jahat.
- **Back-Channel (server-ke-server):** Panggilan jaringan HTTPS langsung antara server backend JalanKita dan server otentikasi Google bersifat privat dan terenkripsi. Jalur ini sepenuhnya melewati browser.

Authorization Code berfungsi sebagai tiket sekali pakai yang berumur sangat pendek (umumnya kurang dari 60 detik). Bahkan jika seorang penyerang berhasil mencuri kode otorisasi dari riwayat URL browser, kode tersebut tidak dapat ditukarkan menjadi access token karena penyerang tidak memiliki `client_secret` milik JalanKita.

Bab 8 — Kedaluwarsa Token dan Refresh Token

Access token sengaja dirancang berumur pendek, umumnya kedaluwarsa setelah satu jam (`expires_in: 3600`). Alasannya sederhana: jika suatu access token bocor, jendela kesempatan bagi penyerang untuk menyalahgunakannya menjadi sangat terbatas, sesuai sisa waktu sebelum token itu kedaluwarsa.

Namun, meminta Rani untuk login ulang dan menekan “Setujui” setiap satu jam sekali tentu akan memberikan pengalaman pengguna yang buruk. JalanKita perlu menyinkronkan kalender secara latar belakang bahkan ketika Rani sedang tidur.

Untuk mengatasi hal ini, OAuth 2.0 memperkenalkan konsep Refresh Token.

8.1 Cara Kerja Refresh Token

Bergantung pada konfigurasi penyedia layanan (misalnya dengan menyertakan parameter `access_type=offline` dan `prompt=consent` untuk Google), Google akan mengembalikan baik `access_token` maupun `refresh_token` berumur panjang pada saat pertukaran kode awal.

JalanKita kemudian mengenkripsi dan menyimpan `refresh_token` tersebut secara aman di basis datanya.

Ketika `access_token` kedaluwarsa, JalanKita melakukan permintaan latar belakang langsung ke endpoint token Google, menyertakan `refresh_token` beserta `client_secret`. Google kemudian memvalidasi refresh token tersebut dan menerbitkan `access_token` baru — tanpa perlu melibatkan Rani sama sekali.

Aspek	Access Token	Refresh Token
Tujuan utama	Mengakses API yang dilindungi	Memperoleh access token baru
Umur	Sangat pendek (15 menit s.d. 1 jam)	Panjang (hari, bulan, atau hingga dicabut)
Dikirim ke mana	Disertakan pada setiap panggilan API ke Resource Server	Hanya dikirim ke endpoint token Authorization Server
Keamanan penyimpanan	Boleh disimpan sementara di memori server	Wajib disimpan terenkripsi di penyimpanan aman

Tabel 8.1 — Perbandingan karakteristik Access Token dan Refresh Token.

Bab 9 — PKCE: Melindungi Public Client

Authorization Code Flow yang telah kita bahas bergantung pada kerahasiaan `client_secret` milik JalanKita. Itulah sebabnya JalanKita, sebagai aplikasi backend yang berjalan di server dan dapat menyimpan variabel lingkungan dengan aman, digolongkan sebagai Confidential Client.

Tetapi bagaimana jika JalanKita berbentuk Single Page Application (misalnya berbasis React atau Vue yang berjalan langsung di browser), atau aplikasi mobile native (iOS/Android)?

Aplikasi semacam ini disebut Public Client. Siapa pun dapat membuka developer tools pada browser atau melakukan dekompilasi berkas .apk Android untuk mengekstrak `client_secret` yang tertanam di dalamnya.

Tanpa adanya rahasia (secret), bagaimana public client dapat menggunakan Authorization Code Flow dengan aman? Jika sebuah aplikasi jahat pada perangkat mobile berhasil mencegat authorization code, aplikasi tersebut berpotensi menukarnya dengan token karena tidak ada `client_secret` yang menghalanginya.

Untuk mengatasi persoalan ini, OAuth 2.0 memperkenalkan PKCE (Proof Key for Code Exchange, dilafalkan “pixie”).

9.1 Cara Kerja PKCE

Sebelum memulai alur, klien menghasilkan sebuah string acak kriptografis yang disebut `code_verifier`. Klien kemudian melakukan hashing terhadap string ini (umumnya menggunakan SHA-256) untuk menghasilkan `code_challenge`.

Pada Langkah 2 dari alur otorisasi, klien mengirimkan `code_challenge` beserta metode hashing-nya (`code_challenge_method=S256`) ke Authorization Server. Authorization Server menyimpan `code_challenge` tersebut dan tetap mengembalikan authorization code seperti biasa.

Pada Langkah 6, saat menukarkan kode dengan token, klien mengirimkan `code_verifier` asli yang belum di-hash.

Authorization Server kemudian melakukan hashing terhadap `code_verifier` yang diterima menggunakan SHA-256, lalu memeriksa apakah hasilnya cocok dengan `code_challenge` yang tersimpan sebelumnya. Jika cocok, ini membuktikan bahwa aplikasi yang meminta token adalah aplikasi yang sama persis dengan yang memulai permintaan tersebut.

Catatan: Pedoman keamanan OAuth modern merekomendasikan penggunaan PKCE untuk semua jenis aplikasi, termasuk aplikasi backend confidential seperti Spring Boot atau Express.js sekalipun.

Bab 10 — state vs. PKCE: Melawan Serangan yang Berbeda

Banyak pengembang mengira parameter state dan mekanisme PKCE adalah hal yang sama karena keduanya sama-sama melibatkan string acak yang dikirim selama alur OAuth berlangsung. Padahal, keduanya melindungi dari jenis serangan yang sepenuhnya berbeda.

Aspek	Parameter state	PKCE (code_verifier)
Ancaman utama	Login CSRF — penyerang menjebak korban agar menyelesaikan alur OAuth menggunakan authorization code milik penyerang.	Pencegatan kode (code interception) — penyerang mencuri authorization code milik korban lalu menukarkannya dengan token.
Cara melindungi	Mengikat callback otorisasi pada sesi browser pengguna yang spesifik.	Membuktikan bahwa entitas yang menukar kode adalah entitas yang sama dengan yang memintanya.
Titik validasi	Diperiksa oleh backend aplikasi klien saat menerima callback.	Diperiksa oleh Authorization Server pada endpoint /token.

Tabel 10.1 — Perbedaan fungsi proteksi antara parameter state dan mekanisme PKCE.

Bab 11 — OAuth 2.0 vs. OpenID Connect (OIDC) dan JWT

Sebagaimana ditekankan pada Bab 2, OAuth 2.0 murni ditujukan untuk otorisasi (izin), bukan autentikasi (identitas). Namun, hampir setiap aplikasi yang kita gunakan sehari-hari memiliki tombol “Masuk dengan Google.” Bagaimana ini bisa terjadi?

11.1 Kehadiran OpenID Connect (OIDC)

OpenID Connect adalah lapisan identitas yang dibangun langsung di atas OAuth 2.0. Jika OAuth murni menerbitkan `access_token` yang ditujukan untuk API, permintaan `scope openid` akan menginstruksikan Authorization Server untuk turut menerbitkan ID Token bersamaan dengan access token.

Ketika JalanKita meminta `scope openid` bersamaan dengan izin kalender:

```
scope=openid profile email https://www.googleapis.com/auth/calendar.events
```

Endpoint token milik Google akan merespons dengan `access_token` DAN `id_token` sekaligus.

- Access Token: ditujukan untuk Resource Server (Google Calendar). JalanKita tidak perlu membaca isinya; token ini cukup diteruskan dalam header permintaan.
- ID Token: ditujukan khusus untuk JalanKita. Token ini berisi informasi tertanda tangan secara kriptografis tentang Rani, misalnya ID pengguna Google-nya, nama lengkap, email, dan URL foto profil.

11.2 Apa Itu JWT?

ID Token hampir selalu diformat sebagai JWT (JSON Web Token, dilafalkan “jot”). JWT adalah string ringkas dan aman-URL yang terdiri dari tiga bagian dipisahkan titik: Header.Payload.Signature.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkJhbmkiLCJpYXQiOiE1MTYyMzkwMjJ9.Sf1KxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Bila bagian tengah (payload) tersebut diuraikan, isinya adalah JSON polos, misalnya:

```
{
  "sub": "google-user-id-98765",
  "iss": "https://accounts.google.com",
  "aud": "JALANKITA_CLIENT_ID",
  "email": "rani@gmail.com",
  "exp": 1711900000
}
```

Encoding Base64URL bukanlah enkripsi. Siapa pun yang memegang sebuah JWT dapat membaca isinya. Namun, karena token tersebut ditandatangani menggunakan private key milik Google, backend JalanKita dapat memverifikasi tanda tangan publik Google secara lokal, tanpa perlu memanggil server Google setiap kali hendak memastikan identitas Rani.

11.3 Apakah Access Token Selalu Berbentuk JWT?

Tidak. OAuth 2.0 sengaja tidak mewajibkan format tertentu untuk access token. Access token dapat berupa:

10. Opaque Token — sebuah string acak sepenuhnya (misalnya `ya29.a0ARdaM...`). Klien dan Resource Server harus mencari tahu maknanya dengan menanyakannya langsung ke Authorization Server.

11. Structured Token (seperti JWT) — berisi klaim (claims) yang tertanam langsung di dalamnya sehingga Resource Server dapat memvalidasinya secara mandiri.

Bab 12 — Mengenal OAuth 2.1

Jika Anda mengikuti pedoman keamanan terkini, Anda akan sering mendengar istilah OAuth 2.1. OAuth 2.1 bukanlah perombakan total atas OAuth 2.0, melainkan sebuah draf konsolidasi yang merangkum praktik-praktik keamanan terbaik selama bertahun-tahun ke dalam satu spesifikasi tunggal.

12. PKCE wajib digunakan. PKCE kini diwajibkan untuk seluruh Authorization Code Flow, termasuk aplikasi sisi server confidential seperti backend Spring Boot atau Express.js.
13. Penghapusan grant type lama. Implicit Grant (yang mengembalikan token langsung melalui URL browser) dan Resource Owner Password Credentials Grant (yang mengumpulkan password secara langsung) dihapus sepenuhnya.
14. Kecocokan redirect URI harus eksak. Penggunaan wildcard pada redirect URI dilarang untuk mencegah eksploitasi open-redirect.

Bab 13 — Kesalahan Keamanan Umum di Lingkungan Produksi

Membangun integrasi OAuth di lingkungan produksi menuntut perhatian yang cermat terhadap detail. Berikut adalah lima kesalahan keamanan paling umum yang sering dilakukan pengembang backend, beserta cara menghindarinya.

13.1 Menyimpan Token di localStorage Browser

Jika Anda membangun klien SPA yang menerima access token atau refresh token, jangan pernah menyimpannya di localStorage atau sessionStorage. Skrip apa pun yang berjalan di halaman Anda — termasuk analitik pihak ketiga, widget chat, atau dependensi npm yang telah disusupi — dapat membaca localStorage melalui celah Cross-Site Scripting (XSS).

Solusi: simpan token di dalam cookie HTTP-Only, Secure, dan SameSite yang dikelola oleh backend Anda, atau gunakan arsitektur backend-for-frontend (BFF) di mana token tidak pernah sampai ke browser sama sekali.

13.2 Membocorkan Client Secret

Terdengar sepele, tetapi string client_secret terus-menerus berakhir di repositori GitHub publik. Ingat: rahasia apa pun yang disertakan dalam aplikasi Android/iOS, aplikasi single-page berbasis React, atau kode frontend lainnya, pada dasarnya bersifat publik.

Solusi: simpan client secret hanya di dalam variabel lingkungan pada environment sisi server. Gunakan PKCE untuk public client yang tidak mampu menjaga kerahasiaan secret.

13.3 Meminta Scope yang Tidak Perlu (Scope Creep)

Meminta akses penuh ke akun pengguna padahal hanya membutuhkan izin baca membuat pengguna curiga dan meningkatkan risiko tanggung jawab jika suatu saat token bocor.

Solusi: terapkan prinsip least privilege (hak akses minimum). Minta hanya scope spesifik yang benar-benar dibutuhkan aplikasi saat itu juga. Jika di kemudian hari JalanKita menambahkan fitur analisis email, mintalah scope Gmail secara dinamis hanya ketika pengguna mengaktifkan fitur tersebut.

13.4 Menganggap Token OAuth Sebagai Bukti Identitas

Sekadar karena sebuah aplikasi menerima access_token dari suatu API, bukan berarti aplikasi tersebut boleh memperlakukan token itu sebagai bukti identitas siapa yang login.

Jika seorang penyerang menyertakan access token valid yang diperoleh dari aplikasi lain (serangan confused deputy), sistem Anda mungkin akan menerimanya begitu saja apabila hanya memeriksa validitas token tanpa memeriksa klaim aud (audience) — yaitu untuk siapa sebenarnya token itu diterbitkan.

Solusi: gunakan OpenID Connect dan validasi klaim pada id_token, termasuk aud dan iss, ketika hendak mengautentikasi pengguna.

13.5 Melewatkan Validasi state atau PKCE

Jika Anda membangun alur OAuth secara manual tanpa memeriksa parameter state, aplikasi Anda menjadi rentan terhadap Login Cross-Site Request Forgery (CSRF). Penyerang dapat menjebak browser korban agar menyelesaikan alur OAuth menggunakan authorization code milik penyerang, sehingga menautkan sesi korban dengan data akun milik penyerang.

Solusi: selalu bangkitkan parameter state yang acak secara kriptografis dan terikat pada sesi pengguna, atau andalkan framework keamanan mapan seperti Spring Security yang sudah menerapkan ini secara otomatis.

Bab 14 — Penutup

OAuth 2.0 bisa terasa sangat rumit jika dipandang semata-mata sebagai kumpulan spesifikasi, RFC, dan istilah teknis. Namun jika lapisan jargon tersebut disingkirkan, OAuth sesungguhnya hanya menyelesaikan satu masalah inti: memungkinkan seorang pengguna memberi izin kepada sebuah aplikasi untuk mengakses datanya, tanpa harus menyerahkan password miliknya.

Setiap komponen dalam protokol ini hadir untuk mendukung misi inti tersebut secara aman:

- Scope membatasi izin yang diberikan.
- Access Token menyediakan akses sementara yang dapat dicabut.
- Authorization Code menjaga token agar tidak melintasi URL browser yang rentan.
- Refresh Token menjaga akses jangka panjang tanpa harus terus-menerus merepotkan pengguna.
- PKCE melindungi aplikasi publik yang tidak dapat menyimpan rahasia.
- OpenID Connect menambahkan lapisan identitas yang terstandardisasi di atasnya.

Lain kali Anda mengintegrasikan penyedia OAuth pada aplikasi backend, atau men-debug kegagalan token di lingkungan produksi, jangan langsung fokus menghafal diagram. Lihatlah permintaan HTTP-nya, tanyakan batas keamanan apa yang sedang dilintasinya, dan pilihan-pilihan desain di baliknya akan langsung masuk akal.

Glosarium

Istilah	Penjelasan
Authorization Code	Tiket sekali pakai berumur pendek yang dikembalikan melalui pengalihan browser, lalu ditukarkan di sisi server dengan access token.
Access Token	Kartu akses sementara yang digunakan pada header HTTP untuk mengakses sumber daya yang dilindungi.
Refresh Token	Kredensial berumur panjang yang digunakan khusus pada endpoint token untuk memperoleh access token baru.
Scope	String yang menentukan izin spesifik yang diminta oleh klien.
PKCE	Proof Key for Code Exchange — teknik kriptografis yang mengikat proses pertukaran token pada instance klien yang memulainya.
OpenID Connect (OIDC)	Lapisan identitas yang dibangun di atas OAuth 2.0, menerbitkan id_token berisi detail profil pengguna.
JWT	JSON Web Token — format ringkas dan bertanda tangan digital yang umum digunakan untuk ID Token.
Client Secret	Kunci rahasia yang digunakan aplikasi sisi server untuk membuktikan identitasnya ke Authorization Server.
Resource Owner	Pengguna yang memiliki dan mengendalikan akses terhadap datanya sendiri.
Bearer Token	Jenis token yang cukup 'dibawa' (disertakan) dalam permintaan untuk mendapatkan akses, tanpa bukti kepemilikan tambahan.

Referensi

Krishna, Ashutosh. "How OAuth 2.0 Works: A Practical Guide for Backend Developers." freeCodeCamp, 29 Juli 2026. Diakses melalui <https://www.freecodecamp.org/news/oauth-2-0-guide-for-backend-developers/>